

Engineering Autonomous Systems with the Claude Agent SDK: Architecture, Production Patterns, and Ecosystem Integration

The rapid evolution of large language models has shifted the paradigm from interactive chat interfaces to fully autonomous, programmatic agents. Anthropic's Claude Agent SDK, formerly known as the Claude Code SDK, represents a critical maturation in this space. By abstracting the complex agent loop, tool orchestration mechanisms, and context compaction algorithms that power the interactive Claude Code CLI, the SDK allows developers to embed frontier models—specifically the Claude 3.5 and 4.5 families—directly into automated pipelines.¹ This analysis provides an exhaustive technical examination of the Claude Agent Python and TypeScript SDKs, exploring core mechanics, production reliability patterns, ecosystem integrations, and advanced architectural strategies for scaling personal automation fleets on systems such as macOS.

Core SDK Mechanics and the API Surface

At the foundational level, the Claude Agent SDK operates differently from traditional, stateless Large Language Model (LLM) APIs. Instead of merely sending a prompt and receiving a string, the SDK initiates a long-running, stateful process capable of executing multi-step reasoning, interrogating environments, and mutating filesystems.³

The Execution Loop and the `query()` API

The primary interface for programmatic execution is the `query()` function. Unlike standard API wrappers that return a single response object, `query()` functions as an asynchronous generator (`AsyncIterator[Message]`), yielding a stream of message objects as the agent iterates through its reasoning loop.¹ The function signature requires a prompt (which can be a string or an asynchronous iterable for streaming input) and accepts an optional `ClaudeAgentOptions` data class.⁴

Because the agent interacts with external systems, the execution loop is inherently unpredictable in duration. As the agent runs, the `query()` generator yields various sub-types of `Message` objects. The structure of these objects is critical for applications that need to monitor the agent's real-time state:

- **AssistantMessage:** This represents the response from the AI and contains `ContentBlock` objects. These blocks can encapsulate raw text (`TextBlock`), internal reasoning for models capable of extended thought (`ThinkingBlock`), or explicit requests to execute tools

(ToolUseBlock), which include the tool's ID, name, and input parameters.⁴ Following a tool call, a ToolResultBlock is yielded, containing the tool's execution status, output content, and any error flags.⁴

- **SystemMessage:** This object contains system-level metadata, such as the initialization event that broadcasts the unique session_id required for resuming conversations.⁴
- **StreamEvent:** Yielded for partial message updates when the include_partial_messages=True parameter is set, allowing developers to render token-by-token streaming in user interfaces.⁴
- **ResultMessage:** The final message yielded when the agent completes its objective or reaches a configured termination threshold. This terminal object encapsulates the final string output, structured data payloads validated against JSON Schemas, execution duration metrics (duration_ms, duration_api_ms), and crucial token usage telemetry.⁴

Alternatively, developers requiring more granular control over the initialization sequence can utilize the ClaudeSDKClient class, which separates the connect(prompt) and receive_messages() phases, allowing for complex asynchronous orchestration before the message stream is consumed.⁴

Configuration via ClaudeAgentOptions

The behavioral boundaries of the agent are strictly defined by the ClaudeAgentOptions object. This comprehensive configuration layer dictates permissions, tool access, budget constraints, and integration pathways.⁵

Security and tool execution are governed by the allowed_tools and disallowed_tools arrays. By default, the SDK includes built-in filesystem tools such as "Read", "Write", "Bash", "Glob", and "Grep".⁶ For autonomous operation without human intervention—such as agents triggered by scheduled cron jobs, GitHub hooks, or macOS launchd—the permission_mode parameter must be explicitly set to "bypassPermissions".¹⁰ Without this configuration, the SDK will halt execution and await standard input from an interactive terminal when attempting sensitive operations like bash command execution, effectively hanging automated background processes.¹²

Financial guardrails are implemented via the max_budget_usd parameter. When set, the SDK continuously calculates the accumulated cost of input and output tokens during the agent loop. If the threshold is breached, the session is aggressively terminated, preventing runaway reasoning loops from causing excessive billing events.¹³ Additionally, the max_turns parameter restricts the absolute number of tool-use iterations the agent can perform, serving as a secondary fail-safe against infinite logical loops caused by persistent tool execution failures.³

Other critical configuration options include fallback_model, which specifies a secondary model to use if the primary API request fails due to rate limits or provider outages, and max_thinking_tokens, which configures the extended reasoning budget for complex

architectural tasks.⁵

Context Management, Prompts, and Filesystem Inheritance

The SDK distinguishes itself through its robust filesystem-based configuration inheritance. By initializing `ClaudeAgentOptions` with `setting_sources=["project"]` in Python or `settingSources: ['project']` in TypeScript, the SDK automatically scans the current working directory for a `.claude/settings.json` file.¹ This capability enables the codebase itself to dictate the agent's behavior, seamlessly bridging the gap between local development with the Claude CLI and production execution via the SDK.

System prompts within the SDK dictate the agent's persona, formatting rules, and operational constraints. By default, the SDK utilizes a minimal system prompt focused purely on tool instructions, omitting project context to save tokens.¹⁶ However, developers can invoke the official CLI behavior by passing `system_prompt={"type": "preset", "preset": "claude_code"}`. This preset injects a comprehensive set of developer guidelines, code styling rules, verbosity settings, and security safety instructions, effectively replicating the exact behavioral profile of the official Anthropic tool.¹⁶

Furthermore, enabling project settings allows the SDK to ingest domain-specific capabilities known as "Skills." Defined as standard Markdown files located in the `.claude/skills/` directory (e.g., `SKILL.md`), these files act as dynamic context injectors.¹ When the `allowed_tools` array includes "Skill", the SDK parses the metadata of these Markdown files upon initialization.¹⁹ The agent is then aware of its available skills and can autonomously invoke them via a tool call when the user prompt demands specialized domain knowledge. This architecture prevents the main system prompt from bloating with irrelevant instructions, loading deep contextual knowledge only when the semantic intent of the task requires it.²⁰ Additional directory-based configurations include Custom Commands located in `.claude/commands/*.md` and overarching project memory managed via the `CLAUDE.md` file.¹

Session Management and State Persistence

A critical architectural consideration when deploying the Claude Agent SDK is its approach to session management. By default, every discrete invocation of the `query()` function is entirely stateless from the perspective of the application layer; it begins a fresh conversation with no historical context.²¹ The SDK does not feature a built-in, cloud-based automatic memory system that tracks users across disparate instantiations.²¹

However, the SDK inherently maintains conversation history locally on the filesystem within a hidden `.claude` directory.³ When a query initiates, the first yielded message is a `SystemMessage` containing a unique UUID `session_id`.⁷ If an application captures this ID, subsequent `query()` calls can be instantiated with the `resume=session_id` parameter.⁷ When resumed, the SDK rehydrates the full conversational context, tool execution history, and filesystem state from the

local disk, restoring the exact state of the agent loop.²¹

Developers can also utilize the `fork_session=True` parameter when resuming.²³ This preserves the original session's immutability while creating a new conversational branch, a pattern highly useful for testing alternative coding implementations or allowing multiple subagents to explore different problem-solving strategies from a common starting point.²⁰

This filesystem-dependent design introduces significant complexities in production environments, particularly when utilizing ephemeral containerization technologies like Docker or Kubernetes. If an agent runs within a standard container, the termination of that container destroys the `.claude` local storage.³ Consequently, any attempt to pass a saved `session_id` to a new container instance will fail to resume the conversation. To achieve true statelessness across distributed deployments, engineering teams must implement persistent volume mounts for the `.claude` directory or design hybrid state hydration architectures that serialize the agent transcript to an external database (such as PostgreSQL or SQLite) before container teardown.³

Production Patterns: Reliability, Cost, and Observability

Transitioning autonomous agents from experimental scripts to production-grade infrastructure requires rigorous attention to error handling, telemetry, and unit economics. When humans are removed from the loop, an agent must rely entirely on programmatic guardrails to survive unexpected environmental states and API inconsistencies.

Intercepting Execution via Agent Hooks

The most powerful mechanism for enforcing deterministic behavior and reliability in the Claude Agent SDK is the Hook system. Configured via the `options.hooks` parameter, hooks allow developers to intercept the agent's internal lifecycle events asynchronously.²⁶ This functionality is crucial for building "Agent Rulez" or deterministic policy engines that govern AI behavior.²⁷

The Python and TypeScript SDKs support several key hook events²⁶:

- **PreToolUse:** Triggers immediately before a tool is executed. This provides a critical intervention window. A hook callback receives the intended tool name and its arguments, and must return a `permissionDecision` of "allow", "deny", or "ask".²⁶ For autonomous agents, this facilitates strict policy enforcement without relying on prompt engineering. For example, a `PreToolUse` hook can evaluate a Bash command request and issue a "deny" decision if the command attempts to modify protected system directories, edit `.env` files, or execute destructive operations like `rm -rf`.²⁶ Furthermore, if the decision is "allow", the hook can actively mutate the execution by returning an `updatedInput` payload, smoothly redirecting file write operations into secure, ephemeral sandbox directories without the agent realizing its output was routed.²⁶

- **PostToolUse:** Triggers after a tool returns its result but before the LLM ingests the output. This is the optimal insertion point for telemetry logging, enabling applications to stream execution events to external observability platforms or trigger asynchronous webhooks.²⁶
- **SubagentStop:** Triggers upon the completion of a delegated subtask. In a multi-agent swarm, this hook acts as an event bus, allowing a master orchestrator to aggregate results, monitor progress, or trigger retry logic if a subagent returns an error state.²⁶

Hook execution is refined using HookMatcher objects, which utilize regex strings to filter specific tool names (e.g., "Bash" or "^mcp_"), ensuring that callbacks only fire for relevant events.²⁶

Cost Optimization: Prompt Caching and Subagent Routing

Unconstrained autonomous agents can consume vast amounts of API credits due to their iterative loops and the inherent bloat of appending file reads and bash outputs to the context window. Beyond the blunt instrument of max_budget_usd, deep cost optimization relies on two primary strategies: Prompt Caching and Subagent Model Routing.

Prompt caching yields substantial savings for long-running sessions, but its mechanics must be precisely understood. When the use_cache=True option is enabled, the Anthropic API retains the initial system prompt, environment context, and historical tool outputs.¹³ However, caching is not automatically applied to all requests; it requires a minimum threshold of tokens to activate.

AI Provider	Frontier Model	Minimum Tokens for Cache Activation
Anthropic	Claude Sonnet 4.5	1,024
OpenAI	GPT-4o / GPT-5.2	1,024
Google	Gemini 2.5 Pro	4,096

As the data demonstrates, Anthropic's models activate caching at a highly efficient threshold of 1,024 tokens.²⁹ To maximize cache hit rates, developers must systematically organize the agent's context. Static elements—such as system instructions, deeply complex JSON schemas for outputFormat, and loaded .claude/skills/SKILL.md files—must be placed at the beginning of the prompt sequence.³⁰ Improper configuration, such as dynamically altering the prefix in every turn or failing to properly load settings, can result in cache write/read inversions, where the system continuously pays to write to the cache without ever reading from it, severely inflating

daily operational costs.¹⁷

Model routing provides the second pillar of cost efficiency, capable of reducing total token expenditure by up to 70%.³¹ The Claude Agent SDK introduces the concept of Subagents, allowing a primary "orchestrator" agent to spawn specialized, isolated subprocesses to handle discrete tasks.³²

A highly effective production pattern involves instantiating the primary orchestrator using a frontier model like Claude Opus 4.5 or Sonnet 4.5 for complex planning, architectural synthesis, and decision-making.³³ When the orchestrator encounters a high-volume, low-complexity task—such as aggressively grepping through thousands of log files, linting code, or scraping web content—it can delegate the task to a designated subagent configured via the `ClaudeAgentOptions` to use a significantly cheaper, faster model like Claude Haiku 4.5.³¹ The Haiku-powered subagent executes its narrow task, compacts the result into a concise summary, and returns it to the orchestrator. This architectural pattern preserves the expensive model's pristine context window, prevents token bloat, and drastically reduces the financial overhead of long-running workflows.³¹

Telemetry, Logging, and Observability

In a headless environment without an interactive terminal, understanding an agent's failure modes requires comprehensive telemetry. The final `ResultMessage` yielded by the SDK exposes a usage dictionary that contains authoritative token counts (`input_tokens`, `output_tokens`).⁴ Because a single SDK query represents an automated ReAct loop consisting of multiple underlying API calls, extracting this cumulative usage data is mandatory for accurate internal billing, cost attribution, and performance profiling.³⁵

For advanced observability, integration with platforms like Langfuse, Datadog, or any OpenTelemetry-compatible collector is essential.³⁶ By capturing traces during the `PostToolUse` hook, developers can map the exact lineage of an agent's decision-making process. This includes tracking the input parameters of tool calls, logging the raw outputs of MCP servers, monitoring execution latencies, and analyzing the frequency of prompt cache hits versus misses.³⁷ Langfuse's v2 Observations API, for instance, allows teams to selectively query usage and metadata field groups to generate dashboards that monitor token accumulation during parallel execution, helping to preemptively trigger context compaction before the agent crashes due to context window exhaustion.³⁸

Testing Autonomous Agents

Testing autonomous systems introduces unique challenges, as network latency, LLM non-determinism, and destructive filesystem operations cause standard unit tests to flake or fail catastrophically. The established pattern for testing the Claude Agent SDK involves integrating with `pytest` and utilizing the `pytest-cldf` plugin alongside `pytest-mock` to construct

deterministic evaluation suites.⁴⁰

To safely test an agent's ability to manipulate a codebase or interact with APIs, developers must isolate the agent from the host machine. This involves injecting mock filesystem paths using temporary directories (tmp_path fixtures in pytest) during test initialization and intercepting external network calls to simulate API responses.⁴⁰ True integration testing requires the creation of structured "evalsets"—standardized repositories representing known application states with specific bugs or missing features.⁴¹ The test suite launches the agent against the evalset, awaits the terminal ResultMessage, and then runs programmatic assertions against the resulting filesystem state to verify that the agent successfully authored the correct code changes without introducing regressions.⁴¹ Best practices dictate that AI-generated tests must always be reviewed, mock data must be kept realistic, and evaluations must measure performance degradation over time.⁴³

Ecosystem and the Competitive Landscape

The release of the Claude Agent SDK has catalyzed the development of numerous third-party wrappers, integrations, and open-source applications, establishing it within a highly competitive landscape of agentic frameworks.⁴⁴

Comparative Analysis of Agent Frameworks

When selecting an agentic foundation, engineering teams must evaluate the Claude Agent SDK against prevailing alternatives based on flexibility, deployment complexity, and abstraction depth.

Framework / SDK	Primary Architecture	Core Strengths	Ideal Use Cases
Claude Agent SDK	Agent Harness (Opinionated)	Production security via Hooks, robust local filesystem utilities, built-in compaction, native MCP support.	Coding assistants, secure personal automation, long-running research tasks. ¹³
OpenAI Agents SDK	Swarm / Multi-Agent	Seamless delegation between lightweight, interconnected	Collaborative reasoning, simple task routing. ⁴¹

		agents.	
LangChain / LangGraph	Agent Framework & Runtime	Maximum ecosystem flexibility, durable execution, complex cyclic graphing.	Highly custom, multi-provider enterprise workflows requiring specific state persistence. ⁴⁵
CrewAI / AutoGen	Low-Code Configuration	Rapid prototyping via simple configuration files and prompts.	Proof-of-concept multi-agent scenarios, users preferring minimal coding. ⁴⁶

As defined by industry practitioners, the Claude Agent SDK operates primarily as an "agent harness" rather than a generalized framework.⁴⁵ It is highly opinionated, optimized specifically for Anthropic's models, and emphasizes deterministic execution. Its core strength lies in its native support for programmatic command execution and its seamless integration with the Model Context Protocol (MCP).¹³ By contrast, LangChain provides broad abstractions over nearly any LLM provider, requiring significant boilerplate code but offering maximum flexibility.⁴⁵ Tools like CrewAI cater to a low-code demographic, excelling in rapid setup but frequently falling short when granular control over the execution loop, raw filesystem access, and error-handling hooks are mandated for mission-critical personal automation.⁴⁶

Open-Source Projects and Ecosystem Tools

The community has rapidly adopted the Claude Agent SDK to build personal productivity tools, knowledge management systems, and specialized infrastructure.

- **Quack:** A native macOS application built with Tauri and Rust that sits on top of the Claude Agent SDK. It implements a visual sidebar for managing multiple parallel agents grouped by project. Crucially, it replaces manual context saving with a "Shared Brain" pattern, utilizing a SQLite database and an MCP server to allow agents to read and write shared context automatically.⁴⁸
- **RecallAgent:** An open-source Slack reminder bot that demonstrates how to build agents that "actually remember." It uses the Claude Agent SDK to process event-triggered webhook payloads from Slack, storing long-term personalization state and memory patterns in an external relational database (Supabase) rather than relying on the ephemeral .claude folder.²⁵
- **Traverse:** A complex multi-agent system built for immigration assistance. It showcases the SDK's ability to coordinate multiple subagents—one scanning social media sentiment, another analyzing proposal data—while using custom MCP tools to stream debate logic in

real-time via WebSockets.⁴⁹

- **Composio:** A prominent wrapper library and tooling provider that extends the SDK's capabilities. Rather than forcing developers to build custom OAuth flows for every external service, Composio provides "meta-tools" and remote MCP servers that handle authentication dynamically, allowing the SDK to securely interact with platforms like GitHub, Jira, and Google Calendar.⁵⁰

Integration Patterns: Scaling the Personal Agent Fleet

Applying the Claude Agent SDK to scale a personal productivity ecosystem—transitioning from a single, schedule-driven daily note creator to a fleet of specialized autonomous agents handling finance, health tracking, and inbox management—demands a fundamental architectural shift. The patterns that succeed for a single, isolated script will rapidly degrade into race conditions, skyrocketing token costs, and corrupted data when scaled to a fleet of concurrent agents.⁴⁸

Rethinking Obsidian Integration: The MCP Gateway

The initial architecture of allowing a single agent to perform direct filesystem Input/Output (I/O) operations against an Obsidian Markdown vault becomes a critical liability at scale.⁴⁸ If a Finance Agent attempts to append a ledger entry to a daily note while the Inbox Agent is simultaneously restructuring the document's frontmatter, the resulting race condition will corrupt the file. Furthermore, reading entire directory structures into the LLM context to locate a specific file causes massive token inflation, quickly exhausting the `max_budget_usd`.⁵¹

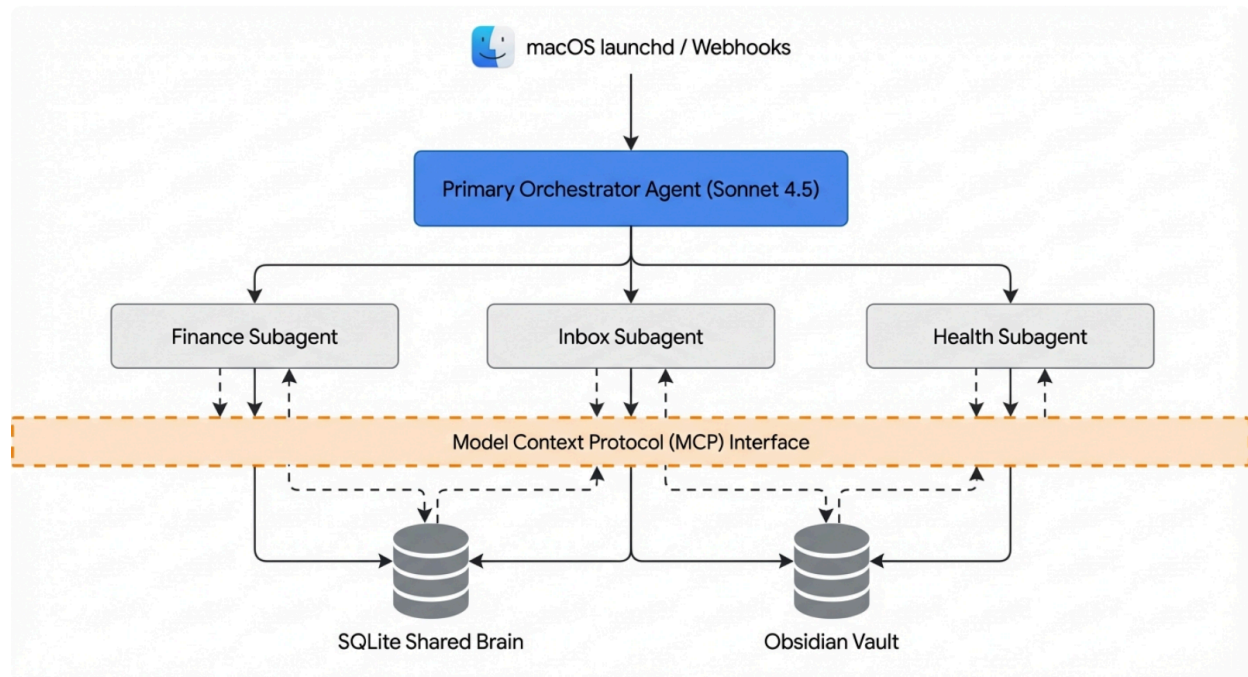
The scalable solution is to entirely revoke direct filesystem access from the agents. Instead, the architecture must transition to utilizing an Obsidian-specific MCP Server (e.g., `obsidian-mcp` or community REST API plugins).¹ By configuring the `ClaudeAgentOptions` to interface with this MCP server, the vault is treated as a remote database. The agents utilize specialized tools exposed by the MCP—such as `search_notes`, `read_note_content`, and `append_to_note`—rather than raw bash commands. This establishes an API gateway pattern, ensuring that the MCP server manages concurrent write locks, provides precise atomic updates, and drastically reduces token consumption by returning only the necessary data payloads rather than entire file contents.⁴⁸

Orchestration: Event-Triggered Subagents vs. Independent Schedules

Deploying six separate scripts bound to six separate launchd cron schedules is an anti-pattern that leads to high latency, redundant initialization costs, and complete context isolation.⁵² If the Inbox Agent identifies a high-priority financial email, the Finance Agent will be completely unaware of it when it executes via cron later in the day. The optimal architectural pattern for a unified personal assistant utilizes the SDK's Subagent capabilities combined with event-driven

triggers.³²

Scalable Multi-Agent Architecture for macOS Automation



To scale personal automation reliably, agents must be decoupled from direct filesystem I/O. A central orchestrator utilizes the Claude Agent SDK to spawn specialized subagents. These subagents communicate with a shared SQLite context database and an Obsidian knowledge vault exclusively through an MCP layer, ensuring thread-safe operations and modular design.

Rather than managing six cron jobs, a single launchd service triggers a master Orchestrator script periodically. This Orchestrator is instantiated with an array of defined agents in its `ClaudeAgentOptions`, each mapped to specific SKILL.md domain expertise.³² Powered by a highly cost-efficient model like Claude Haiku 4.5, the Orchestrator queries the environment (checking external webhooks, scanning for new health data dumps via file watchers) to determine what work is required.³¹

If it detects pending financial operations, the Orchestrator autonomously spawns the Finance Subagent using the built-in Task tool, seamlessly passing it the necessary context.³² The Finance Subagent—configured to utilize the deeper reasoning capabilities of Claude Sonnet 4.5—executes the complex analysis, commits the result to the Obsidian vault via MCP, and terminates, returning control to the Orchestrator.³¹ This hierarchical delegation confines heavy token expenditure strictly to the moments it is required, vastly improving the economic viability

of the entire ecosystem.

The Shared Brain Pattern and Third-Party Authentication

To bridge the gap between discrete, stateless context windows and ensure continuity between the Inbox and Finance subagents, the system must implement a "Shared Brain" pattern. This involves standing up a local SQLite database accessible to all agents via a custom MCP server.⁴⁸

This database acts as an asynchronous message bus and semantic state store. The Inbox Agent writes extracted financial data into a standardized table within the SQLite database. When the Finance Agent is subsequently booted by the Orchestrator, its initial system prompt instructs it to query the SQLite MCP server for unread financial events.⁴⁸ By centralizing cross-agent memory into a highly structured SQL format, the system avoids the token bloat of maintaining massive markdown files for inter-agent communication while ensuring perfect transactional integrity.

Finally, connecting these subagents to external platforms like Jira, Google Calendar, and Slack requires careful authentication management. Passing raw API keys directly into the SDK's context window exposes the keys to the LLM and risks leakage during logging. The modern integration pattern relies on remote MCP servers or frameworks like Composio.⁵⁴ These frameworks provide "meta-tools" (e.g., `COMPOSIO_SEARCH_TOOLS`, `COMPOSIO_MULTI_EXECUTE_TOOL`) that act as a proxy layer.⁵⁰ The agent requests an action, and the framework executes the API call using OAuth tokens securely managed outside the agent's process. This abstracts the complexities of OAuth refresh tokens and pagination away from the LLM, allowing the subagent to focus entirely on the logical execution of its designated task.⁵⁰

Conclusion

The Claude Agent SDK provides a highly deterministic, programmatic wrapper around the formidable capabilities of Anthropic's frontier models. It shifts the developmental focus from managing the minutiae of LLM inference strings to engineering robust system architectures capable of long-term, autonomous operation.

For the transition from a single automation script to a highly available, multi-agent fleet on macOS, the architectural imperatives are clear. Relying on direct filesystem mutations and disparate cron jobs will yield diminishing returns in reliability and exponential increases in token costs. Success at scale mandates the adoption of the Model Context Protocol to encapsulate local data stores like Obsidian, the implementation of a SQLite-backed shared memory bus to hydrate context across ephemeral execution windows, the use of execution hooks to enforce strict behavioral policies, and the strategic deployment of orchestrator-subagent hierarchies to aggressively manage inference economics. By adhering to these production patterns, developers can harness the Claude Agent SDK to build highly resilient, intelligent systems

capable of continuous background execution without human intervention.

Works cited

1. Agent SDK overview - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/overview>
2. Claude Agent SDK: Build Agents That Work Like Claude Code - DEV Community, accessed February 22, 2026, <https://dev.to/rajeshroyal/claude-agent-sdk-build-agents-that-work-like-claude-code-50ln>
3. Hosting the Agent SDK - Claude API Docs - Claude Console, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/hosting>
4. Agent SDK reference - Python - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/python>
5. 0.1.0 • pypi-claude-agent-sdk • tessel • Registry, accessed February 22, 2026, <https://tessel.io/registry/tessel/pypi-claude-agent-sdk/0.1.0/docs/configuration-options.md>
6. CLAUDE_AGENT_SDK_SPEC.md - GitHub Gist, accessed February 22, 2026, <https://gist.github.com/SamSaffron/603648958a8c18ceae34939a8951d417>
7. Session Management - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/sessions>
8. Agent SDK reference - TypeScript - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/typescript>
9. Get structured output from agents - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/structured-outputs>
10. One drive MCP Integration with Claude Agent SDK | Composio, accessed February 22, 2026, https://composio.dev/toolkits/one_drive/framework/claude-agents-sdk
11. The Perl Claude Agent - DEV Community, accessed February 22, 2026, <https://dev.to/lnationorg/the-perl-claude-agent-1ipk>
12. claude-agent-sdk-tool-integration - LobeHub, accessed February 22, 2026, <https://lobehub.com/ru/skills/thebushidocollective-han-tool-integration>
13. securevibes/docs/references/claude-agent-sdk-guide.md at main - GitHub, accessed February 22, 2026, <https://github.com/anshumanbh/securevibes/blob/main/docs/references/claude-agent-sdk-guide.md>
14. claude_agent_sdk_rs - Rust - Docs.rs, accessed February 22, 2026, <https://docs.rs/claude-agent-sdk-rs>
15. Claude Code - Truefoundry Docs, accessed February 22, 2026, <https://www.truefoundry.com/docs/ai-gateway/claude-code>
16. Modifying system prompts - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/modifying-system-prompts>
17. Cache Control in SDK · Issue #89 · anthropics/claude-agent-sdk-typescript - GitHub, accessed February 22, 2026, <https://github.com/anthropics/claude-agent-sdk-typescript/issues/89>

18. Agent Skills - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>
19. building-with-claude-agent-sdk | Ski... - LobeHub, accessed February 22, 2026, <https://lobehub.com/tr/skills/panaversity-agentfactory-building-with-claude-agent-sdk>
20. The Claude Developer Guide Agent SDK Reference - GoPenAI, accessed February 22, 2026, <https://blog.gopenai.com/the-claude-developer-guide-agent-sdk-reference-guides-92c0c2f3b3ef>
21. open-docs/docs/claude-agent-sdk/memory-and-context.md at main - GitHub, accessed February 22, 2026, <https://github.com/bgauryy/open-docs/blob/main/docs/claude-agent-sdk/memory-and-context.md>
22. How to Save State in Claude Agents SDK? : r/Al_Agents - Reddit, accessed February 22, 2026, https://www.reddit.com/r/Al_Agents/comments/1p4h6eq/how_to_save_state_in_claude_agents_sdk/
23. Claude Agent SDK - Promptfoo, accessed February 22, 2026, <https://www.promptfoo.dev/docs/providers/claude-agent-sdk/>
24. Forking of existing sessions - Agent Client Protocol, accessed February 22, 2026, <https://agentclientprotocol.com/rfds/session-fork>
25. Building Memory-First AI Reminder Agents with Mem0 and Claude Agent SDK, accessed February 22, 2026, <https://mem0.ai/blog/building-a-reminder-agent-that-actually-remembers>
26. Intercept and control agent behavior with hooks - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/hooks>
27. Agent RuleZ: A Deterministic Policy Engine for AI Coding Agents - Medium, accessed February 22, 2026, <https://medium.com/@richardhightower/agent-rulez-a-deterministic-policy-engine-for-ai-coding-agents-9489e0561edf>
28. Build Agent Skills Faster with Claude Code 2.1 Release | by Rick Hightower - Medium, accessed February 22, 2026, <https://medium.com/spillwave-solutions/build-agent-skills-faster-with-claude-code-2-1-release-6d821d5b8179>
29. Don't Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks, accessed February 22, 2026, <https://arxiv.org/html/2601.06007v2>
30. Don't Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks, accessed February 22, 2026, <https://arxiv.org/html/2601.06007v1>
31. Claude Code Sub-agents: The 90% Performance Gain Nobody Talks About, accessed February 22, 2026, <https://www.codewithseb.com/blog/claude-code-sub-agents-multi-agent-systems-guide>
32. Subagents in the SDK - Claude API Docs, accessed February 22, 2026, <https://platform.claude.com/docs/en/agent-sdk/subagents>
33. Deploy and use Claude models in Microsoft Foundry (preview), accessed

February 22, 2026,

<https://learn.microsoft.com/en-us/azure/ai-foundry/foundry-models/how-to/use-foundry-models-claude?view=foundry-classic>

34. A Guide to Claude Code 2.0 and getting better at using coding agents | sankalp's blog, accessed February 22, 2026,
<https://sankalp.bearblog.dev/my-experience-with-claude-code-20-and-how-to-get-better-at-using-coding-agents/>
35. Tracking Costs and Usage - Claude API Docs, accessed February 22, 2026,
<https://platform.claude.com/docs/en/agent-sdk/cost-tracking>
36. Trace with OpenTelemetry - Docs by LangChain, accessed February 22, 2026,
<https://docs.langchain.com/langsmith/trace-with-opentelemetry>
37. LangChain Tracing & LangGraph Integration, accessed February 22, 2026,
<https://langfuse.com/integrations/frameworks/langchain>
38. Observations API - Langfuse, accessed February 22, 2026,
<https://langfuse.com/docs/api-and-data-platform/features/observations-api>
39. [BUG] No mid-loop compaction for custom MCP tools; parallel tool batches skip compaction check · Issue #531 · anthropics/claude-agent-sdk-python - GitHub, accessed February 22, 2026,
<https://github.com/anthropics/claude-agent-sdk-python/issues/531>
40. pytest Documentation, accessed February 22, 2026,
<https://media.readthedocs.org/pdf/pytest/latest/pytest.pdf>
41. The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents - arXiv, accessed February 22, 2026,
<https://arxiv.org/html/2511.03690v1>
42. CLAUDE.md - coleam00/remote-agentic-coding-system - GitHub, accessed February 22, 2026,
<https://github.com/coleam00/remote-agentic-coding-system/blob/main/CLAUDE.md>
43. I Built a \$0/month Autonomous QA Agent That Writes Tests for My Team Using Claude Code + Self-Hosted GitLab - Reddit, accessed February 22, 2026,
https://www.reddit.com/r/ClaudeCode/comments/1oun4gp/i_built_a_0month_autonomous_qa_agent_that_writes/
44. How to build AI agents with MCP: 12 framework comparison (2025) - ClickHouse, accessed February 22, 2026,
<https://clickhouse.com/blog/how-to-build-ai-agents-mcp-12-frameworks>
45. Agent Frameworks, Runtimes, and Harnesses- oh my! - LangChain Blog, accessed February 22, 2026,
<https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
46. Top 10 tools to build AI Agents (most recent) : r/AI_Agents - Reddit, accessed February 22, 2026,
https://www.reddit.com/r/AI_Agents/comments/1qbzqkv/top_10_tools_to_build_ai_agents_most_recent/
47. Langchain is OpenAI Agents SDK and both are Fundamental Orchestration - Reddit, accessed February 22, 2026,
https://www.reddit.com/r/LangChain/comments/1jblqcp/langchain_is_openai_age

[nts_sdk_and_both_are/](#)

48. Claude Code + Obsidian - How I use it & Short Guide : r/ClaudeAI - Reddit, accessed February 22, 2026,
https://www.reddit.com/r/ClaudeAI/comments/1qr19df/claude_code_obsidian_how_i_use_it_short_guide/
49. Gallery | Built with Opus 4.6: a Claude Code hackathon - Cerebral Valley, accessed February 22, 2026,
<https://cerebralvalley.ai/e/claude-code-hackathon/hackathon/gallery>
50. I'm Building My Own Coding Agent Harness (And It's Pretty Cool) - DEV Community, accessed February 22, 2026,
<https://dev.to/composiodev/im-building-my-own-coding-agent-harness-and-its-pretty-cool-1lpf>
51. How to Turn Claude Code Into Your Personal AI Assistant - The Neuron, accessed February 22, 2026,
<https://www.theneuron.ai/explainer-articles/how-to-turn-claude-code-into-your-personal-ai-assistant/>
52. From Traces to Evaluation: An Autonomous MLflow Agent Harness for AI AgentOps - Databricks Community, accessed February 22, 2026,
<https://community.databricks.com/t5/technical-blog/from-traces-to-evaluation-a-n-autonomous-mlflow-agent-harness-for/ba-p/143422>
53. Effective harnesses for long-running agents - Anthropic, accessed February 22, 2026,
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
54. How to Connect to Live Google Calendar Data from Claude Agent SDK (via CData Connect AI), accessed February 22, 2026,
<https://www.cdata.com/kb/tech/googlecalendar-cloud-claude-agent-sdk.rst>